HEIG-VD HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Prof. Yann Thoma

Formal verification
Verification of an elevator model
CERN training - November 2026

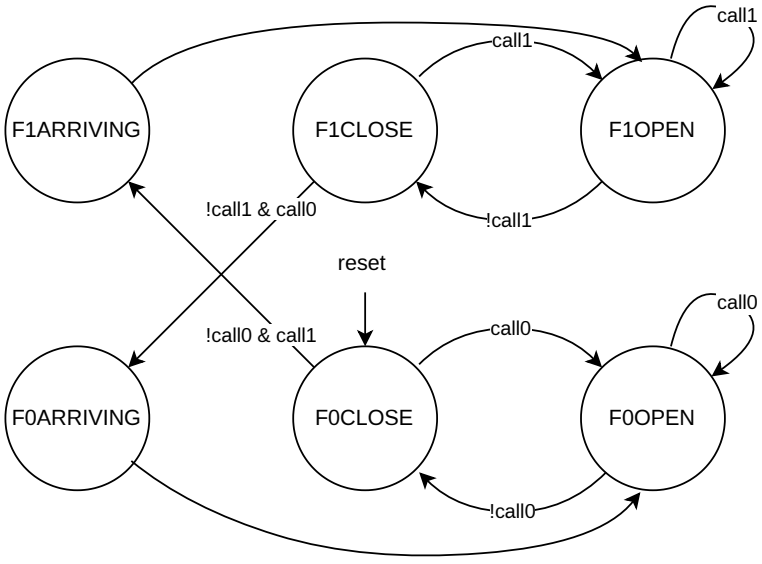
Context

We want to verify the correctness of a finite-state machine (FSM) that models the behavior of an elevator.
The entity declaration is as follows :

```
entity elevator_fsm is
    port(
        clk_i    : in  std_logic;
        rst_i    : in  std_logic;
        call0_i  : in  std_logic;
        call1_i  : in  std_logic;
        open_o   : out std_logic;
        floor0_o : out std_logic;
        floor1_o : out std_logic
        );
end elevator_fsm;
```

First version

The two inputs are used to call the elevator to floor 0 or floor 1. A call is accepted only when the door is closed and the elevator is waiting on a floor. Calling the elevator from the floor it is already on simply opens the door. The door remains open as long as the button for that floor is pressed.
When the elevator travels to another floor, it passes through a state in which it is arriving at that floor. In this state, both floor outputs are low because the elevator is not yet at either floor.
The asynchronous reset places the elevator at floor 0 with the door closed.
The state machine corresponds to the following graph :



`open_o` is `'1'` in states `F0OPEN` and `F1OPEN`, and must be `'0'` in all other states.
`floor0_o` is `'1'` in states `F0CLOSE` and `F0OPEN`, and must be `'0'` in all other states.

`floor1_o` is `'1'` in states `F1CLOSE` and `F1OPEN`, and must be `'0'` in all other states.
Add assertions to the `.psl` file to verify that the FSM behaves correctly. First, use only the input and output ports. Then, because you also have access to the architecture and the FSM state, add assertions to check the state transitions.
A generic parameter, `ERRNO`, can be used to inject errors into the design. Its behavior is as follows :

1. When `ERRNO` is 0, the design behaves correctly.
2. When `ERRNO` is between 1 and 6, inclusive, the design contains an error.

Use different values of `ERRNO` to test your assertions.
You can change its value in the `.sby` file.
Use the code in folder `v1`.

Second version

To make the elevator FSM more realistic, we changed its behavior so that the door remains open for 10 clock cycles. If the button for the current floor is pressed while the door is open, the 10-cycle period restarts from the time the button is pressed.
The corresponding code is in the `v2` folder. You can start by copying your previous assertions, as some of them should still hold, and then modify them as needed.
A generic parameter, `ERRNO`, can be used to inject errors into the design. Its behavior is as follows :

1. When `ERRNO` is 0, the design behaves correctly.
2. When `ERRNO` is between 1 and 6, inclusive, the design contains an error.

Third version

The design team decided that the FSM should not contain a counter, so they moved the counter outside the FSM. To support this change, they added two ports :

— `counter_init_o`, an output that initializes the counter to 0;
— `counter_done_i`, an input that indicates when the counter has reached its final value (for example, after 10 clock cycles).

The FSM must respond to the `counter_done_i` signal and drive `counter_init_o` correctly.
The `counter_init_o` signal is expected to be high just before the FSM enters one of the *OPEN* states, and low at all other times.
The door should also close when `counter_done_i` goes high.
The corresponding code is in the `v3` folder. You can start by copying your previous assertions, as some of them should still hold, and then modify them as needed.
Keep in mind that the FSM, as currently implemented, does not allow the door to remain open for only one clock cycle.
A generic parameter, `ERRNO`, can be used to inject errors into the design. Its behavior is as follows :

1. When `ERRNO` is 0, the design behaves correctly.
2. When `ERRNO` is between 1 and 6, inclusive, the design contains an error.